מערכות הפעלה תרגיל ב'

1. הבעיה נגרמת כאשר מספר תהליכונים (agents) מנסים לגשת ולעדכן את הקובץ המשותף בו זמנית ללא סנכרון מתאים. זה גורם למצב שבו אחד מהתהליכונים עשוי לקרוא או לכתוב בזמן שתהליכונים אחרים עושים את אותו הדבר, מה שגורם לאי-עקיבות בנתונים.

הרצת הקוד לפני תיקון המראה על אי עקביות:

```
----- Controler -----
Sum of A1 = -382
Sum of A2 = -100
Sum of A3 = -199
Sum of A4 = -239

Not consistent! Somebody steals?!
Controler: -420, Account: -41
Terminating all agents!
tehila2003@DESKTOP-OPO0EU0:~$
```

הינה הקוד המתוקן (צרפתי רק את החלקים שבהם נעשה שינוי וכל השאר כפי שניתן בתרגיל)

```
tehila2003@DESKTOP-OPO0EU0: ~
  GNU nano 6.2                          agents.c
#include <semaphore.h>

#include "agents.h"

int init (int);
int check_point(int*);
void go_agent (char *);
int log_sum (char *);
int update(int, char *, char * );
int write_to_log(int, char *);
void fatal_error(char *,char *);
int gen_rand(int);

/* Global array for agent names */
char agents[MAX_AGENTS][NAMES_LEN] = AGENTS;
int p; /* number of the started agents */
pthread_t thread[MAX_AGENTS], cThread; /* agent threads, controller thread */

/* Semaphore for synchronizing access to the account file */
sem_t sem;

/* Reset all files, Dispatch all agents and control actions */
int main(int ac, char **av)
{
    int i = 0, num_agents = 1; /* default */
    if (ac == 2)  i = atoi(av[1]);  /* Check arguments */
    else if (ac != 1) fatal_error("USAGE: agents [number_of_agents]\n", NULL);

    /* Initialize semaphore */
    sem_init(&sem, 0, 1);

    /* Main process distributes work to threads (agents).
       Agents perform independent operations. Every agent updates the account file
       after each operation and logs them in a personal log file */
    for (i = 0, p = 0; i < num_agents; i++, p++) {
        if (pthread_create(&thread[p], NULL, (void *)go_agent, &agents[p])) {
            perror("ERROR creating thread."); --p;
        }
    }

    i = p;
    printf("Finished creating %d agent(s).\n", num_agents);
    sleep(1);

    /* The father waits for agents to return successfully */
    while (p >= 1) { sleep(2); }

    /* Check the consistency (log files vs. the account file) */
    check_point(&i);

    /* Destroy semaphore */
    sem_destroy(&sem);

    return 0;
}
```

```c
/* Send agent to work */
void go_agent(char *agent_name)
{
    int n, i, amount;

    for (i = 0; i < OPERATIONS; ++i) {
        amount = gen_rand(MAX_OPER);

        sem_wait(&sem); /* Lock semaphore before updating account */

        if (update(amount, ACCOUNT, agent_name) == ERROR) {
            fatal_error("Can't Update Account File", ACCOUNT);
        } else {
            if (write_to_log(amount, agent_name) == ERROR) { /* rollback */
                if (update(-amount, ACCOUNT, NULL) == ERROR) {
                    fprintf(stderr, "ERROR: Rollback failed. Agent %s\n", agent_name);
                }
                fatal_error("Can't log operations", agent_name);
            }
        }

        sem_post(&sem); /* Unlock semaphore after updating account */

        printf("Op of %s = %d\n", agent_name, amount);

        if ((n = rand() % MAX_SLEEP) != 0) sleep(n);
    }
```

הסבר שינויים:

הוספת ספריית sem.h: נוספה הספרייה לצורך שימוש בסמפור.

הכרזת סמפור גלובלי: הוגדר סמפור גלובלי בשם sem.

אתחול והשמדת סמפור: סמפור מאותחל בתחילת התוכנית ומושמד בסופה.

סינכרון גישה לקובץ החשבון: נעשה שימוש בפונקציות sem_wait ו- sem_post לצורך סינכרון גישה לקובץ החשבון.

זה הרצה של הקוד המתוקן:

```
 ----- Controller -----
Sum of A1 = -166
Sum of A2 = -179
Sum of A3 = -150
Sum of A4 = -172

Operations O.K. Controller: -167, Account: -167
tehila2003@DESKTOP-OPO0EU0:~$
```

2. הבקר מבצע בדיקות עקביות בזמנים קבועים מראש, מה שמגביל את היכולת שלו לזהות בעיות בזמן אמת. כאשר יש מספר סוכנים הפועלים במקביל, עשויה להתרחש אי-עקביות במידע הנכתב בקובץ המשותף (account) וביומנים האישיים של הסוכנים. הבקר עלול לפספס את אי-העקביות אם היא מתרחשת מחוץ לחלון הזמן הקבוע שבו הוא מבצע את הבדיקה.

חלקי הקוד המתוקנים:

```c
int main(int ac, char** av)
{
    int i = 0, num_agents = 1; /* default */
    if (ac == 2)  i = atoi(av[1]);  /* Check arguments */
    else if (ac != 1) fatal_error("USAGE: agents [number_of_agents]\n", NULL);

    /* Initialize semaphore */
    sem_init(&sem, 0, 1);

    /* Initiate an account file with a start sum */
    num_agents = init(i);

    /* Main process distributes work to threads (agents).
       Agents perform independent operations. Every agent updates the account file
       after each operation and logs them in a personal log file */
    for (i = 0, p = 0; i < num_agents; i++, p++) {
        if (pthread_create(&thread[p], NULL, (void*)go_agent, &agents[p])) {
            perror("ERROR creating thread."); --p;
        }
    }

    /* Create the controller thread */
    if (pthread_create(&cThread, NULL, controller_thread_func, &num_agents)) {
        perror("ERROR creating controller thread.");
    }

    i = p;
    printf("Finished creating %d agent(s).\n", num_agents);
    sleep(1);

    /* The father waits for agents to return successfully */
    while (p >= 1) { sleep(2); }

    /* Destroy semaphore */
    sem_destroy(&sem);

    return 0;
```

```c
/* Check if agents' actions are consistent */
int check_point(int* num_agents)
{
    int i, n, control_sum, sum;

    control_sum = 0;
    if ((n = rand() % MAX_SLEEP) != 0) sleep(n * 3);

    printf("\n\n ----- Controller ----- \n");

    if ((sum = update(0, ACCOUNT, NULL)) == ERROR) return(ERROR);

    for (i = 0; i < *num_agents; ++i) {
        control_sum += log_sum(agents[i]);
    }

    control_sum += START_SUM;

    if (sum != control_sum) {
        fprintf(stderr, "\nNot consistent! Somebody steals?!\n");
        fprintf(stderr, "Controller: %d, Account: %d \n", control_sum, sum);
        fprintf(stderr, "Terminating all agents!\n");

        /* Terminate all agent threads */
        for (i = 0; i < *num_agents; ++i) {
            pthread_cancel(thread[i]);
        }
        exit(ERROR);
    }

    printf("\nOperations O.K. Controller: %d, Account: %d\n", control_sum, sum);
    return 0;
}
```

```c
/* Controller thread function to check if agents' actions are consistent */
void* controller_thread_func(void* arg)
{
    int num_agents = *((int*)arg);
    while (1) {
        sleep(rand() % MAX_SLEEP + 1);  // Random sleep before checking

        if (check_point(&num_agents) == 1) {
            break;
        }
    }
    return NULL;
}
```

כדי לאפשר לבקר לבצע בדיקות בזמנים אקראיים, בצעתי את השינויים הבאים:

יצירת תהליכון עבור הבקר: במקום שהבקר יפעל כפעולה אחת בודדת בסיום עבודת הסוכנים, ניצור עבורו תהליכון נפרד שיבצע בדיקות בזמנים אקראיים במהלך הפעילות של הסוכנים.

בדיקות בזמנים אקראיים: התהליכון של הבקר יבצע בדיקות בזמנים אקראיים, על ידי שימוש בפונקציית sleep עם ערכים רנדומליים. כך, הבקר יוכל לבדוק את העקביות של הקובץ המשותף והיומנים האישיים של הסוכנים במהלך הפעילות ולא רק בסופה.

סיום התוכנית במקרה של אי-עקביות: אם הבקר יגלה אי-עקביות, הוא יסיים את כל התהליכים של הסוכנים ויפסיק את הריצה של התוכנית.

3.פונקציית pthread_join() משמשת להמתנה לסיום תהליכונים. שאלה זו שואלת האם זה רעיון טוב למקם אותה בסוף הפונקציה הראשית להמתנה לסיום כל התהליכונים. התשובה היא שבדרך כלל זה רעיון טוב, אך יש לקחת בחשבון שיש לוודא שכל התהליכונים באמת צריכים לסיים את עבודתם לפני שהפונקציה הראשית מסיימת.

```
int main(int ac, char** av)
{
    int i = 0, num_agents = 1; /* default */
    if (ac == 2)  i = atoi(av[1]);  /* Check arguments */
    else if (ac != 1) fatal_error("USAGE: agents [number_of_agents]\n", NULL);

    /* Initialize semaphore */
    sem_init(&sem, 0, 1);

    /* Initiate an account file with a start sum */
    num_agents = init(i);

    /* Main process distributes work to threads (agents).
       Agents perform independent operations. Every agent updates the account file
       after each operation and logs them in a personal log file */
    for (i = 0, p = 0; i < num_agents; i++, p++) {
        if (pthread_create(&thread[p], NULL, go_agent, &agents[p])) {
            perror("ERROR creating thread."); --p;
        }
    }

    /* Create the controller thread */
    if (pthread_create(&cThread, NULL, controller_thread_func, &num_agents)) {
        perror("ERROR creating controller thread.");
    }

    printf("Finished creating %d agent(s).\n", num_agents);
    sleep(1);

    /* The father waits for agents to return successfully */
    for (i = 0; i < num_agents; ++i) {
        pthread_join(thread[i], NULL);
    }

    /* Join the controller thread */
    pthread_join(cThread, NULL);

    /* Destroy semaphore */
    sem_destroy(&sem);
```